```cpp
#include <iostream>


using namespace std;



    int main() {

    /*int number{ 10 };

    int* ptr = nullptr;

    ptr = &number;

    cout << "the location of number is " << &number << endl << endl;

    cout << "The location of the ptr is" << ptr << " The ptr derefrenced
      is: " << *ptr << endl << endl;

    ptr++;

    cout << "The location of the ptr is" << ptr << " The ptr derefrenced
      is: " << *ptr << endl << endl;

    double* ptr1 = nullptr;

    double money = 19.99;

    ptr1 = &money;


    cout << "The location of the ptr is" << ptr1 << " The ptr derefrenced
      is: " << *ptr1 << endl << endl;

    ptr1++;


    cout << "The location of the ptr is" << ptr1 << " The ptr derefrenced
      is: " << *ptr1 << endl << endl;*/

    /*



    ptr++;  // Moves 4 bytes to new addy

    ptr = 100; // Cant assign a value to a ptr only location

```

```
46      ptr += 100; // ptr = ptr + 100 moves 400 bytes to different addy
47
48      *ptr = 100; // Derefrenced and assigns ptr to point to new value
49
50      +++*ptr; // Pre works think order of operations, moves addy forward
          then defreences based on data type
51
52      (*ptr)++; // quatations to isolate order of operations;
53
54      */
55
56
57          /* int* ptr = new int // (HEAP) dynamic allocation
58              ptr = new int
59              int* ptr = new int[6]
60
61              //The heap delete where your ptr is pointing
62              // delete ptr; // delete []ptr; // deletes your variable in
                  the heap not your ptr
63              //Data clears not location//
64              // makes ptr's reusable
65              // new is its own specific operator, heap allocation
66              // makes it possible to ask user for size of an array
67
68
69              // dot operator // Access public functions from specific
                  classes
70
71
72              // .length() // string class//
73
74              //PUBLIC MEMBER VARIABLE // ATTRIBUTES OF A CLASS
75
76          */
77
78
79          // PTR CLASS NOTES//
80
81          //int size{};
82
83
84          //cout << "What size array would you like";
85          //cin >> size;
86
87
88          //int* ptr = nullptr;
89          //int* ptr1 = nullptr;
90
91          //ptr1 = ptr;
```

```
92
93          //ptr = new int[size];
94
95
96
97
98          //cout << "Enter: " << size << "Integers.\n";
99
100         //for (int i = 0; i < size; i++) {
101
102         //  cout << "Enter " << i + 1 << " Value: ";
103         //  cin >> *ptr;
104
105         //  ptr++;
106
107
108         //}
109
110         //cout << "\n\n Here are the values from the array\n";
111         ///*ptr--;
112         //ptr--;
113         //ptr--;
114         //....*/
115
116         //// ptr -= size; // or numeric expression depending on the program
117
118         //for (int i = 0; i < size; i++) {
119
120         //  cout << *ptr1 << "\t";
121
122         //  ptr1++;
123
124         //}
125
126
127         //cout << endl << endl;
128
129
130         ////REVERSE
131
132
133         //for (int i = 0; i < size; i++) {
134
135         //  ptr1--;
136
137         //  cout << *ptr1 << "\t";
138
139         //
```

```
140
141         //}
142
143
144 //for (int i = 0; i < size; i++) {
145 //
146 //  cout << ptr[i] << "\t"; // basically derefrenced
147 //
148 //
149 //
150 //}
151
152
153
154
155
156
157
158
159
160
161
162
163
164     return 0;
165 }
166
167
168
169
170
171
172
173
```